`LiGrad`: User Manual

Author: Eleftherios Voulimiotis

Draft v2.3 — July 20, 2026

Abstract

This manual documents the function `grav_dark_transit_model` from the Python code `LiGrad`, created in the context of my bachelor thesis "Transit Model for Gravity-Darkened Oblate Stars" (Voulimiotis, 2025). In this document, we first explain the logical steps that the code follows to generate the transit light-curves of planets orbiting oblate-rotating stars with gravity darkening. Next, we describe the code processes and functions that execute these logical steps, along with the main user interface (expected units for each input parameter, recommended usage, example script).

Contents

1 Installation

1.1 Python dependencies

The package depends on the following Python libraries:

- `numpy` (Harris et al., 2020)
- `scipy` (Virtanen et al., 2020)
- `pylightcurve` (Tsiaras et al., 2016)
- `astropy` (Astropy Collaboration et al., 2022)

Additional libraries used for some example scripts in GitHub are `matplotlib` (Hunter, 2007), `exoclock` (Kokori et al., 2022), `emcee` (Foreman-Mackey et al., 2013) and `astroquery` (Ginsburg et al., 2019).

1.2 Install from pip (PyPI)

Published to PyPI:

```
pip install ligrad
```

1.3 Clone from GitHub

Clone directly from GitHub:

```
git clone https://github.com/evoulimiotis/ligrad.git
cd ligrad
python setup.py install
```

2 Code Description

The main objective of my thesis was the development of new code that is able to fully simulate the transit light-curves of planets orbiting oblate stars with gravity darkening. In this section we will talk about the `LiGrad` code (Transit Light-Curves for Gravity-Darkened Oblate Stars) which was constructed for this phenomenon, and how it works. The code was made entirely in Python and requires certain libraries in order to run properly (dependencies).

2.1 Main Inputs and Function

The main function has the following form:

```
grav_dark_transit_model(t_vals, orbital_period, st_mass,
                  st_mean_radius, st_mean_temperature, beta, lamda,
                  i_s, omega, u1, u2, e, i_0, omega_p, raan, t_p, rp_rs,
                  obs_wavelength=800e-9, integration_grid_size=1)
```

Here, `t_vals` is an array of time data (in days), `period` is the orbital period (days), M, $R_{\rm mean}$, $T_{\rm mean}$ are the stellar mass, mean radius and mean effective temperature relative to the solar values (for example if the stellar mass-radius is $1.7M_{\odot}$-$1.7R_{\odot}$, then the given argument should be 1.7 in both cases). The gravity-darkening exponent is β. The code also requires the sky-projected obliquity λ and the stellar inclination i_s, the rotation factor ω (extracted from catalog stellar rotational velocities $u_{eq} \sin i_s$) and the quadratic limb-darkening coefficients u_1, u_2. Finally, the orbital elements (eccentricity e, orbital inclination i_0, argument of periapsis ω_p, right ascension of ascending node Ω and time of periastron t_p are inputs. The planet size is given as $R_p/R_\star$, which is the radius normalized to the star's mean projected radius.

In short, the function terms and their units are as follows:

```
t_vals,               # ndarray: times at which to compute flux (days)
orbital_period,       # float: orbital period (days)
st_mass,              # float: stellar mass (in solar masses)
st_mean_radius,       # float: stellar radius (in solar radii)
st_mean_temperature,  # float: in units of 10000 K
beta,                 # float: gravity-darkening exponent (dimensionless)
lamda,                # float: spin-orbit angle (degrees)
i_s,                  # float: stellar spin inclination (degrees)
omega,                # float: stellar rotation parameter (dimensionless)
u1,u2,                # floats: Claret 4-term limb-darkening coefficients
e,                    # float: orbital eccentricity (0 to 1)
i_0,                  # float: orbital inclination (degrees)
omega_p,              # float: argument of periastron (degrees)
raan,                 # float: right ascension of ascending node (degrees)
t_p,                  # float: time of periastron / reference epoch (in days)
rp_rs,                # float: planet radius relative to effective stellar radius
obs_wavelength=800e-9,   # float: observation wavelength (meters)
integration_grid_size=1  # int: occulted integration grid resolution
```

- **t_vals, orbital_period, t_p**: **days**. Use the same time system consistently. If you want to work in BJD_TDB or similar, ensure all times are in that system.
- **st_mass**: **solar masses** (the code multiplies input by $M_\odot$).
- **st_mean_radius**: **solar radii** (the code multiplies input by $R_\odot$).
- **st_mean_temperature**: **input in units of 10,000 K**.
  Reason: normalization for data fitting. The code multiplies the `st_mean_temperature` argument by 10000 internally. Example: for a star with 6000 K, pass `st_mean_temperature = 0.6`.
- **lamda**, **i_s**, **i_0**, **omega_p**, **raan**: **degrees**. (Within the code `lamda` and `i_s` are converted to radians before geometric rotations.)
- **omega**: dimensionless rotation factor. It is used to estimate the polar-equatorial radius through $R_p = R_{\text{eq}} \cdot \frac{2}{2+\omega^2}$ inside the code. The values are in the range $0 \leq \omega < 1$.
- **u1, u2**: dimensionless quadratic limb-darkening coefficients.
- **rp_rs**: ratio of planetary radius to effective stellar radius (dimensionless). The code computes a physical planet radius in the projected stellar coordinate units as `Rp_phys = rp_rs * R_eff`.
- **obs_wavelength**: SI meters, with default 800 nm = 800e-9 m, as an average value of the TESS (Transiting Exoplanet Survey Satellite) bandpass. This works as a monochromatic approximation.
- **integration_grid_size**: integer controlling integration points used in the occulted-flux integration (larger values result to better accuracy but makes the total runtime slower).

2.2 Spheroid and Coordinates

The star is modeled as an oblate spheroid. The code uses an equatorial radius R_{eq} (set to the mean radius in the function call) and computes a polar radius R_p through the dimensionless rotation parameter ω. We create the spheroidal surface as a uniform grid of points given in spherical coordinates (drawn from the Fibonacci sequence using the golden angle) and then convert them to 3D Cartesian coordinates.

After generating the surface points, the code rotates them to the observer frame using rotation matrices. The input angles i_s and λ are essentially converted into two rotation angles, and these rotations are applied so that the star is seen in the correct sky-projected orientation.

2.3 Point Visibillity

To determine which surface points "face" the observer and which are on the invisible side, the code calculates the gradient $\vec{\eta}$ of each point and then checks the dot product of $\vec{\eta}$ with the unit vector of the x-axis $\hat{i}$. If $\vec{\eta} \cdot \hat{i} > 0$ then the point is considered visible and we keep it, removing ones with $\vec{\eta} \cdot \hat{i} < 0$.

2.4 Temperature, Gravity Darkening and Intensity

In the code, we compute the effective local gravity at each surface point. Then the local temperature is set by the power law:

$$T(\theta) = T_{\text{mean}} \left(\frac{g(\theta)}{g_{\text{mean}}} \right)^{\beta} ,$$

The stellar intensity is calculated from the Planck function. This intensity is later multiplied by the limb-darkening factor which will be described next.

For simplicity, the code uses a single wavelength at $800nm$ instead of integrating over a real bandpass (for example the TESS telescope has a bandpass of $600 - 1000nm$), so the resulting fluxes are monochromatic approximations to what an instrument would see.

2.5 Stellar 2-D Projected Ellipse

The general sky-projected shape of the spheroid is an ellipse. This indicates that the radius of the projected object is not constant. So, we must find a way to calculate its outer radius, in order to compute the limb darkening. To accomplish this, the code fits an ellipse model equation to the points on the projected ellipse boundary and computes the radius as a function of the polar angle around the center. Then, it sorts points by angle and interpolates the radial distance. This lets the code test if an arbitrary point on the (y, z) plane is inside the stellar disk by comparing its distance to the interpolated edge radius at that angle.

2.6 Limb Darkening

The code finds the projected center and radius of the star (using the previous method). Then, it defines the normalized radial coordinate ρ and $\mu = \sqrt{1 - \rho^2}$ and applies the 4-parameter Claret limb-darkening law:

$$I_{\mathrm{LD}}(\mu) = 1 - u_1(1 - \mu) - u_2(1 - \mu)^2 \,,$$

setting the intensity to zero outside the projected disk. Multiplying the Planck intensity by this limb-darkening factor, gives us the final intensity assigned to each visible surface point.

2.7 Unocculted Flux

The total flux from the visible stellar disk is computed once per unique set of stellar parameters. The visible surface points $(y_{\mathrm{vis}}, z_{\mathrm{vis}})$ are obtained by masking the rotated spheroid with the visibility condition. The projected area of the visible disk is estimated from the convex hull of those points, and a uniform area element is assigned as:

$$dA = \frac{A_{\mathrm{proj}}}{N_{\mathrm{vis}}} \,,$$

where N_{vis} is the number of visible points. The total unocculted flux is then:

$$F_0 = \sum_{i=1}^{N_{\mathrm{vis}}} I_i \, dA \,,$$

with $I_i = I_{\mathrm{grav}}(\theta_i) \cdot I_{\mathrm{LD}}(y_i, z_i)$ being the total intensity at point i.

2.8 Planet Position

The planet's position is found from the orbital elements for each point in time. The code uses Kepler's third law to get the semi-major axis and then calls the module `pylightcurve` to generate the orbit. We assume the planet to be a light blocker (no emission). Then, we consider only the orbital positions in which the planet is located at positive x-coordinates. If $x < 0$, the planet begins traveling towards the backside of the star and the flux is set to 1 (no occultation).

2.9 Occulted Flux

The occulted flux at each transit time step is computed by placing a uniform grid (again drawn by the Fibonacci sequence) of $n \times n$ points centered on the planet position (y_p, z_p), spanning $\pm R_p$ in both directions. Grid points are filtered by two masks applied in a specific order:

1. **Planet disk mask**: Keeps only points satisfying $(Y - y_p)^2 + (Z - z_p)^2 \leq R_p^2$.
2. **Stellar disk mask**: For each remaining point, computes its polar angle $\varphi = \arctan 2(Z - z_c,\, Y - y_c)$ with respect to the fitted projected ellipse center (y_c, z_c), then interpolates the ellipse boundary radius $r(\varphi)$ from a precomputed array of 100 angles, and keep only points with $\sqrt{(Y - y_c)^2 + (Z - z_c)^2} \leq r(\varphi)$.

For these filtered grid points, the nearest precomputed stellar surface point is found with a K-D tree, and its intensity I_{total} is assigned to the grid point. The occulted flux is then:

$$\Delta F = \sum_j I_j \, dA_p \ , \qquad dA_p = \frac{\pi R_p^2}{n} \, ,$$

2.10 Time Loop

The computation of the spheroid point distribution, rotations, visibility mask, ellipse fit, intensity profile, and K-D tree depends only on the stellar parameters. Therefore, all of these quantities are computed once and reused across all time steps. The K-D tree of the visible projected points $(y_{\text{vis}}, z_{\text{vis}})$ is built outside the transit loop, reducing each time step to a fast nearest neighbor search. The most expensive part of the code is the calculation of the planetary position and the occulted flux (planet integration) $\Delta F(t)$, because they are evaluated at every time step. So, after all these steps the code finally returns the normalized flux:

$$\frac{F}{F_0} = 1 - \frac{\Delta F}{F_0}$$

2.11 Internal Subfunctions

Below is a short description of the subfunctions that executes the previous steps:

`spheroid(Re_eq, Rp_polar, n)`: Builds a uniform spheroidal mesh (latitude-longitude) with equatorial radius `Re_eq` and polar radius `Rp_polar`, using the Fibonacci sequence. Returns surface points (x, y, z) and the polar angle grid `V` used later for gravity calculations.

`rotation_matrix_x(angle)` / `rotation_matrix_y(angle)` / `rotation_matrix_z(angle)`: Basic 3D rotation matrices.

`rotated_spheroid(x,y,z,lamda,i_s)`: Rotates the spheroid to align the stellar spin axis according to the projected spin-orbit angle `lamda` and stellar inclination `i_s`.

`vis_mask(x, y, z, Re_eq, Rp_polar, R)`: Computes the surface normals of the spheroid surface points, used to decide visibility (positive dot products mark the visible hemisphere).

`ellipse_radius(y_vis, z_vis, theta)`: Given a 2-D projection of an ellipse, finds the convex hull and returns radial distances from the center to the hull as an interpolated function of angle. This is used to compute the effective projected stellar radius and normalized radial coordinates.

`gravity_darkening(omega, Re_eq, Rp_polar, theta)`: Computes the local effective gravity and applies the gravity-darkening law $T_{\rm local} = T_{\rm mean}(g/g_{\rm mean})^{\beta}$. It then computes an intensity from the Planck law at the observed monochromatic wavelength (`obs_wavelength`).

`limb_darkening(y_proj, z_proj, u1,u2,u3,u4)`: Computes the Claret 4-parameter limb darkening factor on the projected surface ($\mu = \sqrt{1-\rho^2}$, where ρ is normalized radius).

`planet_position(t,period,e,inc,w,t_p)`: Wrapper that computes planetary coordinates at time `t` using `pylightcurve.planet_orbit`. The code computes a semi-major axis from the input stellar mass and period (Kepler's third law) and passes parameters to `pylightcurve`.

`unocculted_flux(I_total, y_vis, z_vis)`: Estimates the total stellar flux by integrating the total visible intensities over the projected stellar disk. The projected area is approximated from the convex hull of the visible coordinates ($y_{\rm vis},\ z_{\rm vis}$).

`occulted_flux(...)`: Computes the stellar flux blocked by the transiting planet, by integrating intensities over a grid covering the planet disk. Only grid points overlapping the projected stellar disk contribute, with intensities taken from the nearest stellar surface elements.

2.12 Performance and Accuracy

- **Precomputed:** The stellar quantities are only computed once for a given set of parameters to avoid costly recomputations when performing the main code loops.
- **integration_grid_size:** Increase to improve accuracy of the planet integration during the occulted flux calculation. Default value of 1 (ok for simulations). Doubling increases the computational cost slightly, but gives better resolution for data fitting purposes. By default, the code runtime is in the range of $0.02-0.2$ sec, depending on computer capabilities-specs.

3 Extra Functions

3.1 Rotational Velocity to dimensionless ω

The function `vsini2omega` converts the rotational velocity at the equator of the star multiplied by the sine of the inclination, to the dimensionless rotational parameter ω. To do that, we use the fixed-point numerical method ($x = g(x)$ for root finding) and calculate the value of ω for an accuracy up to 10^{-9}.

`vsini2omega(vsini_kms, st_mass_solar, R_mean_solar, i_s_deg):` It converts the observed projected stellar rotational velocity, $v \sin i$, into the dimensionless stellar rotation rate, ω, expressed as a fraction of the critical (Keplerian break-up) rotation rate. It takes as input the measured $v \sin i$, the stellar mass, the mean stellar radius, and the inclination of the stellar rotation axis. The true equatorial rotational velocity is first obtained by correcting the observed $v \sin i$ for the inclination angle. Since the stellar equatorial radius depends on the rotation rate through rotational flattening, the critical angular velocity also changes with rotation. Consequently, the calculation is performed iteratively until a self-consistent value of the dimensionless rotation rate is reached. This parameter is defined as:

$$\omega = \frac{\Omega}{\Omega_{\rm crit}} \,,$$

where Ω is the stellar angular rotation rate and $\Omega_{\rm crit}$ is the critical angular velocity. This dimensionless parameter is subsequently used to determine the stellar oblateness and the gravity-darkening intensity distribution across the stellar surface.

4 Examples

4.1 Example Python script

```
import numpy as np
import matplotlib.pyplot as plt
from ligrad import grav_dark_transit_model

t_vals = np.linspace(-0.12, 0.12, 400)
orbital_period = 3.0
st_mass = 1.0
st_mean_radius = 1.0
st_mean_temperature = 0.6
beta = 0.25
lamda = 30.0
i_s = 60.0
omega = 0.8
u1, u2 = 0.3, 0.1
e = 0.0
i_0 = 89.0
omega_p = 0.0
raan = 0.0
t_p = 0.0
rp_rs = 0.1
obs_wavelength = 800e-9

transit_flux = grav_dark_transit_model(t_vals, orbital_period, st_mass,
                    st_mean_radius, st_mean_temperature, beta, lamda,
                    i_s, omega, u1, u2, e, i_0, omega_p, raan, t_p,
                    rp_rs, obs_wavelength=obs_wavelength,
                    integration_grid_size=1)

plt.figure(figsize=(10,6))
plt.plot(t_vals, transit_flux)
plt.xlabel("Time from epoch (mid-transit in days)")
plt.ylabel("Normalized flux")
plt.title("Example Transit")
plt.grid(True, alpha=0.25)
plt.tight_layout()
plt.show()
```

4.1.1 Plotted Output (example figure)

Below is the plotted transit light-curve from the script above:

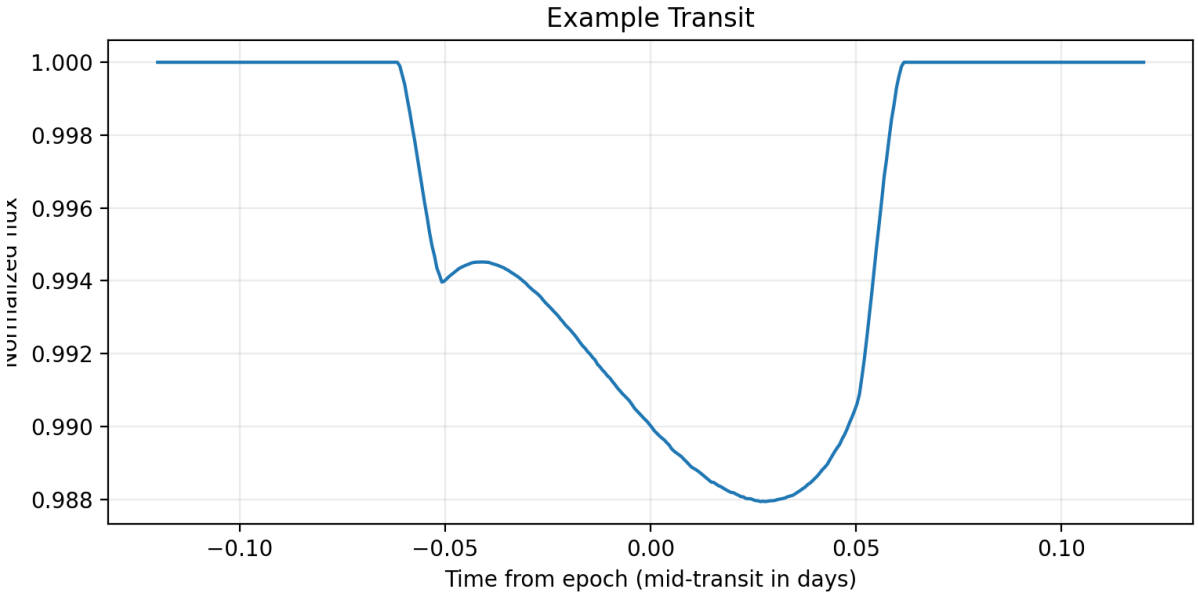
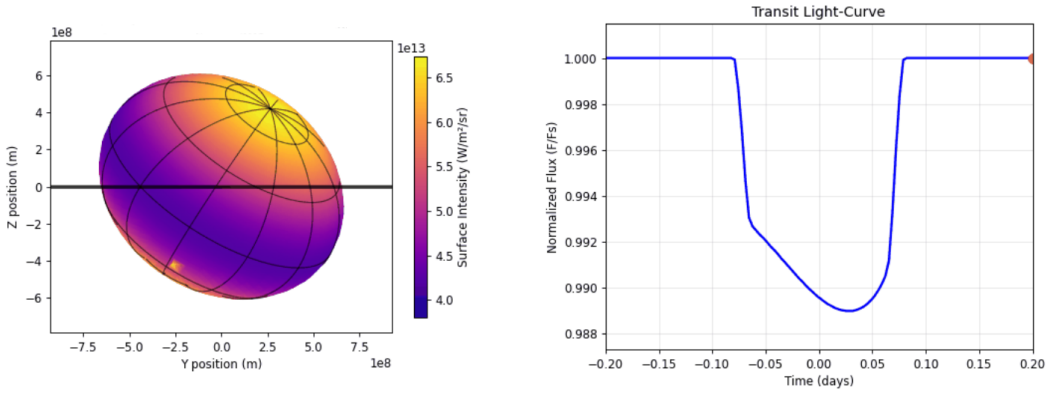


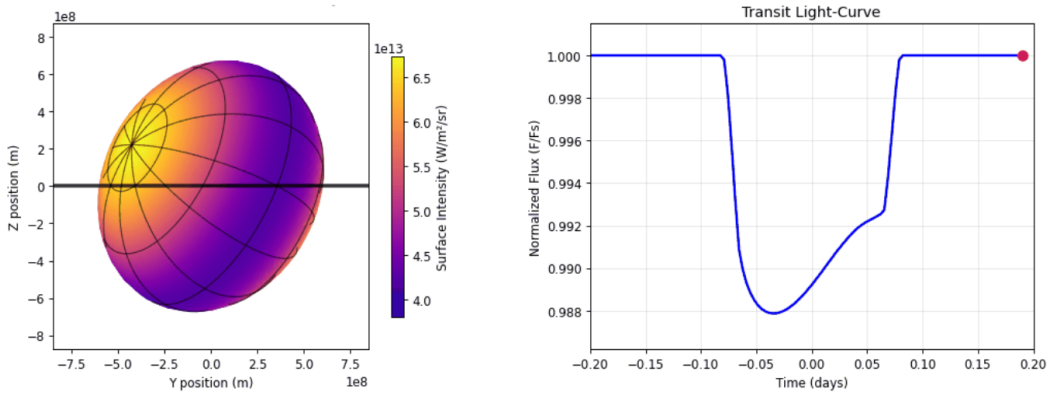
Figure 1: Simulated transit light-curve with example script

4.1.2 Example Simulations

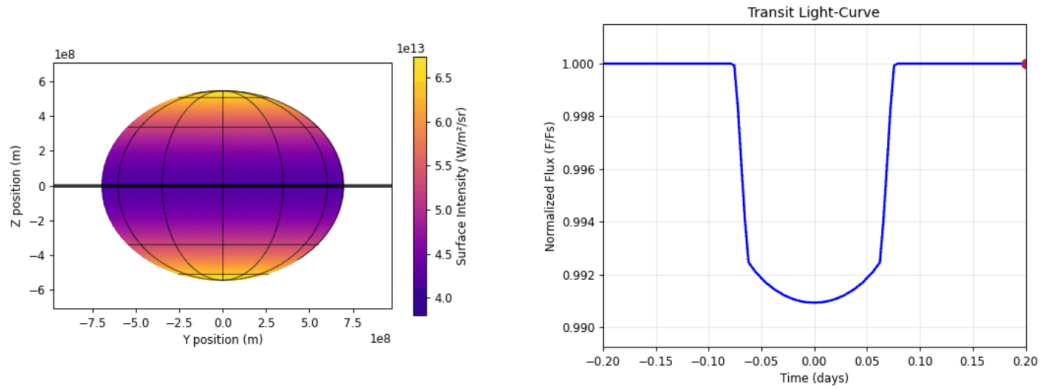
Using the code, we also generated test light-curve models for different set of parameters. First of all, for a rotation factor of $\omega = 0.75$, and a gravity-darkening exponent of $\beta = 0.15$, the parameters varied to alter the light-curves were the geometric terms λ, i_s, i_0. The rest of the parameters are kept constant with values $P = 10\,\text{days}$, $M = M_\odot$, $R_{mean} = R_\odot$, $T_{mean} = 9000\,\text{K}$, u_1, u_2, u_3, $u_4 = 0.5, -0.1,\ 0.05,\ 0.01$, $e = 0$, $\omega_p = 0$, $t_p = 0$, $R_p/R_\star = 0.1$. The light-curves can be seen below:



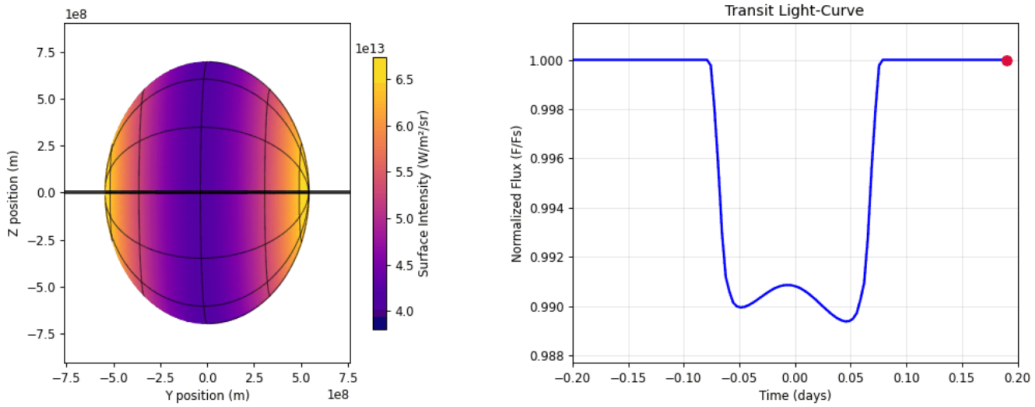
(a) Simulated transit light-curve with $\lambda = 45°,\ i_s = 60°,\ i_0 = 90°$).



(b) Simulated transit light-curve with $\lambda = 285°,\ i_s = 60°,\ i_0 = 90°$).



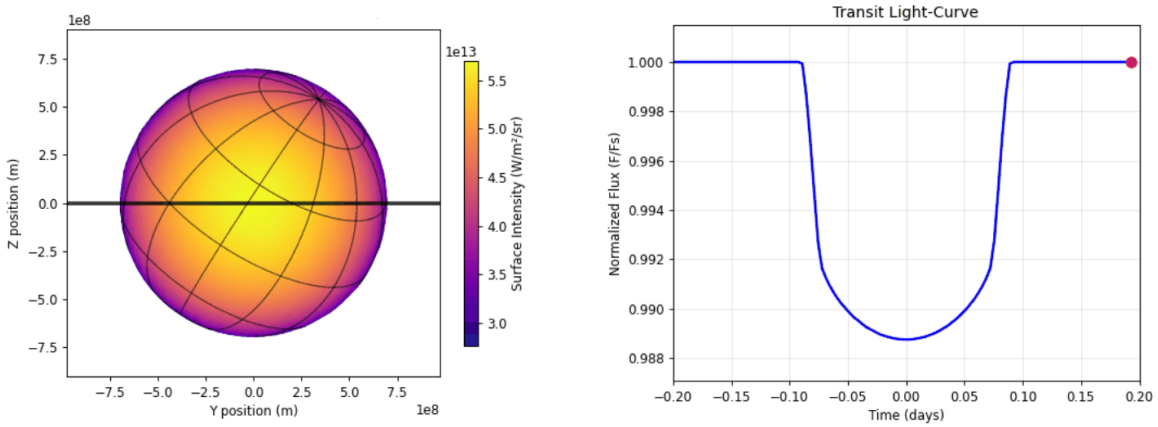
(c) Simulated transit light-curve with $\lambda = 0°,\ i_s = 90°,\ i_0 = 90°$).



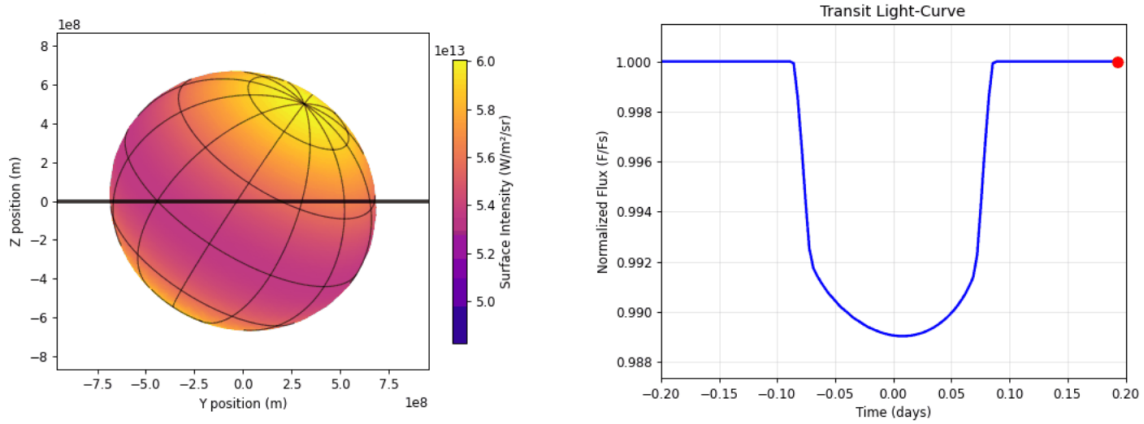
(d) Simulated transit light-curve with $\lambda = 90°,\ i_s = 90°,\ i_0 = 90°$).

Figure 2: Simulated transit light-curves for different sets of angles $\lambda,\ i_s,\ i_0$).

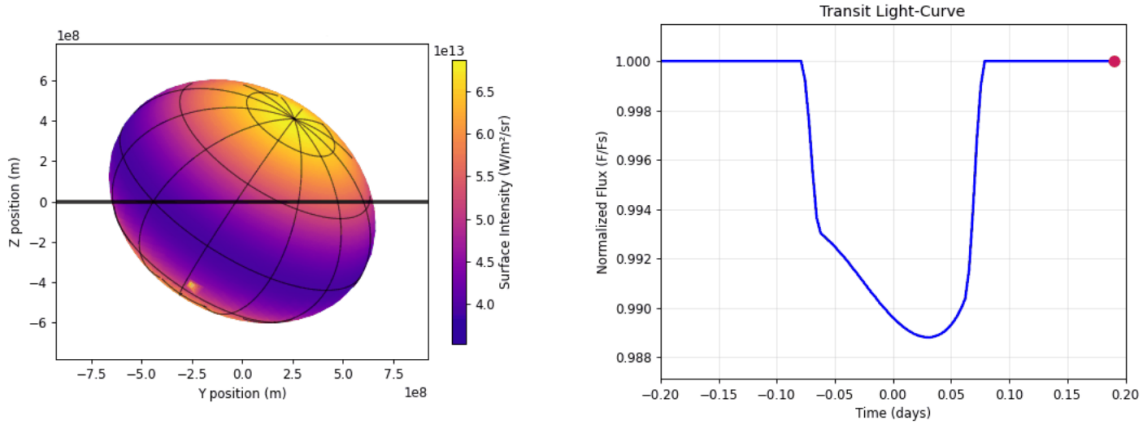
The second set of graphs is mainly based on the stellar parameters β and ω. We consider a static geometry of $\lambda = 45°,\ i_s = 60°,\ i_0 = 90°$, as well as the same constant parameters of the previous simulations ($P = 10\,\text{days},\ M = M_\odot,\ R_{mean} = R_\odot,\ T_{mean} = 9000\,\text{K},\ u_1,\ u_2,\ u_3,\ u_4 = 0.5, -0.1,\ 0.05,\ 0.01,\ e = 0,\ \omega_p = 0,\ t_p = 0,\ R_p/R_\star = 0.1$). The new light-curves are the following:



(a) Simulated transit light-curve with $\beta = 0.15,\ \omega = 0$.

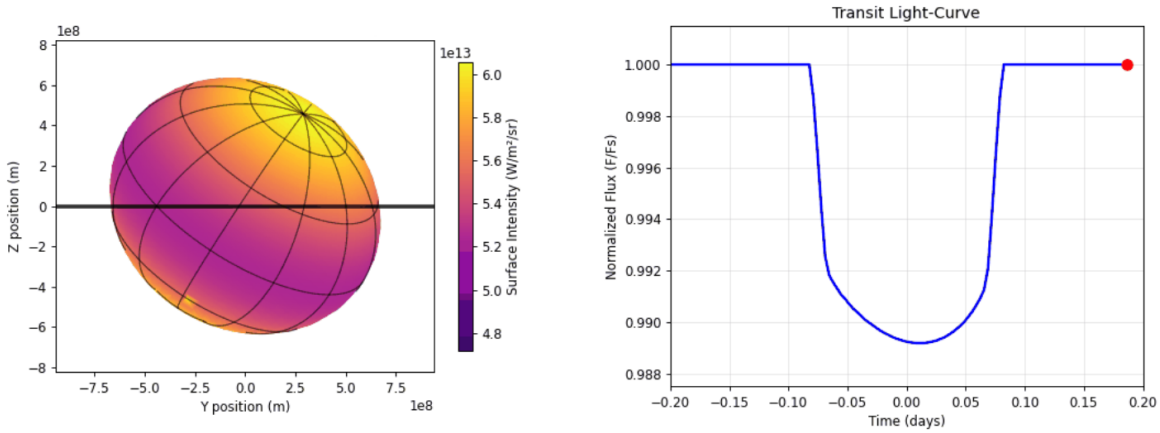


(b) Simulated transit light-curve with $\beta = 0.15,\ \omega = 0.4$.

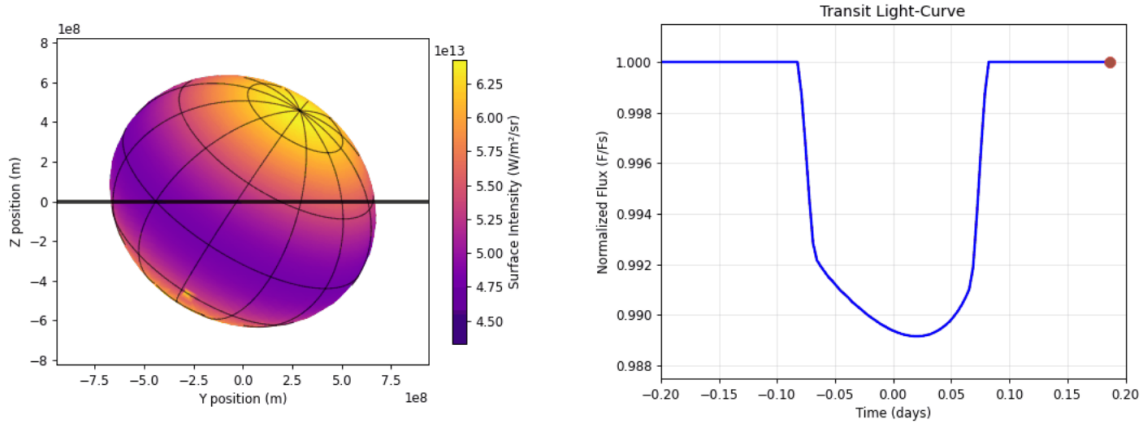


(c) Simulated transit light-curve with $\beta = 0.15,\ \omega = 0.8$.

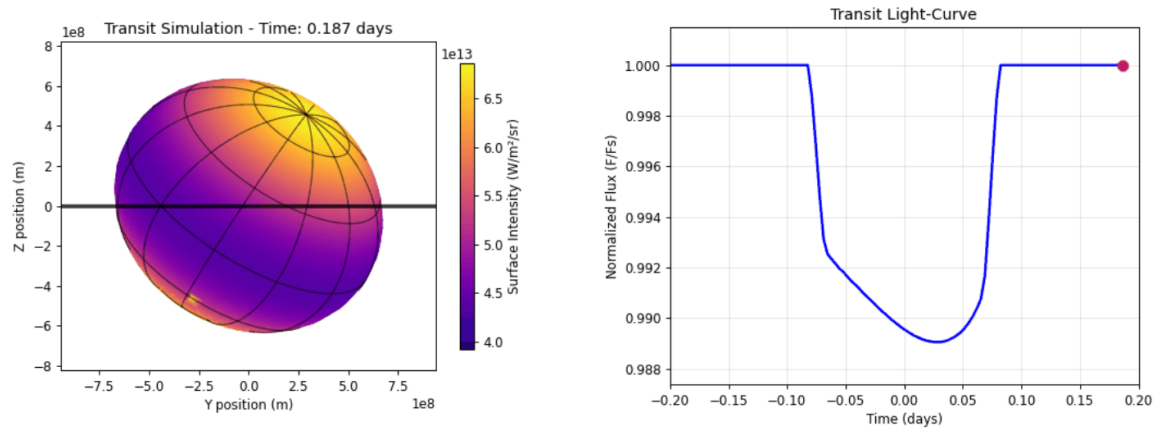
Figure 3: Simulated transit light-curves for different values of ω.



(a) Simulated transit light-curve with $\beta = 0.08,\ \omega = 0.6$.



(b) Simulated transit light-curve with $\beta = 0.16,\ \omega = 0.6$.



(c) Simulated transit light-curve with $\beta = 0.25,\ \omega = 0.6$.

Figure 4: Simulated transit light-curves for different values of β.

References

Astropy Collaboration, Price-Whelan, A. M., Lim, P. L., Earl, N., Starkman, N., Bradley, L., Shupe, D. L., Patil, A. A., Corrales, L., Brasseur, C. E., Nöthe, M., Donath, A., Tollerud, E., Morris, B. M., Ginsburg, A., Vaher, E., Weaver, B. A., Tocknell, J., Jamieson, W., ... Astropy Project Contributors. (2022). The Astropy Project: Sustaining and Growing a Community-oriented Open-source Project and the Latest Major Release (v5.0) of the Core Package., *935*(2), Article 167, 167. https://doi.org/10.3847/1538-4357/ac7c74

Foreman-Mackey, D., Hogg, D. W., Lang, D., & Goodman, J. (2013). emcee: The MCMC Hammer., *125*(925), 306. https://doi.org/10.1086/670067

Ginsburg, A., Sipőcz, B. M., Brasseur, C. E., Cowperthwaite, P. S., Craig, M. W., Deil, C., Guillochon, J., Guzman, G., Liedtke, S., Lian Lim, P., Lockhart, K. E., Mommert, M., Morris, B. M., Norman, H., Parikh, M., Persson, M. V., Robitaille, T. P., Segovia, J.-C., Singer, L. P., ... a subset of the astropy collaboration. (2019). astroquery: An Astronomical Web-querying Package in Python., *157*, Article 98, 98. https://doi.org/10.3847/1538-3881/aafc33

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hunter, J. D. (2007). Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, *9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Kokori, A., Tsiaras, A., Edwards, B., Rocchetto, M., Tinetti, G., Wünsche, A., Paschalis, N., Agnihotri, V. K., Bachschmidt, M., Bretton, M., Caines, H., Caló, M., Casali, R., Crow, M., Dawes, S., Deldem, M., Deligeorgopoulos, D., Dymock, R., Evans, P., ... Tomatis, A. (2022). ExoClock project: an open platform for monitoring the ephemerides of Ariel targets with contributions from the public. *Experimental Astronomy*, *53*(2), 547–588. https://doi.org/10.1007/s10686-020-09696-3

Tsiaras, A., Waldmann, I. P., Rocchetto, M., Varley, R., Morello, G., Damiano, M., & Tinetti, G. (2016, December). *pylightcurve: Exoplanet lightcurve model*.

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., ... SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, *17*, 261–272. https://doi.org/10.1038/s41592-019-0686-2

Voulimiotis, E. (2025). *Transit model for gravity-darkened oblate stars* [Bachelor's Thesis]. Aristotle University of Thessaloniki, Faculty of Sciences, School of Physics [GRI-2025-51194]. https://ikee.lib.auth.gr/record/368128/